Permutation branching programs for AND reading one variable at most twice need length at least four n minus four

James Cook, Ian Mertz

doi:10.4230/LIPIcs.CCC.2022.8; CC BY 4.0

Editorial note. Complete original published author theorem and proof faithfully transcribed under CC BY4. All needed definitions and local support proofs retained. Literal original theorem/lemma numbers and expressions retained, without mathematical corrections. Illustrative Figure3 is not a dependency of this proof; the actual permutation model and transformations are fully editable. Production and validation only onYupeng viaSSH.

Allowed standard prerequisites (editorial).

- Elementary permutation group identities and branching programs. All source-local permutation transformations, restriction and cyclic-rotation lemmas retained with full original author proofs; no external nonstandard proof invoked

4.1 Notation and tools

Before going into our results, we formally define permutation branching programs along with the notation we will use in the rest of this section. These programs will have a more specialized form than when we introduced them in Section2, which we subsequently justify. We assume basic familiarity with permutations, and we write $\sigma_1\sigma_2$ as a shorthand for $\sigma_2 \circ \sigma_1$.

Definition17. Let $n, m, s \in \mathbb{N}$ and let $f : \{0,1\}^n \to \{0,1\}$ be a function such that $f(0 \dots 0) = 0$. A permutation branching program is a sequence $P = \pi_1 \dots \pi_\ell$, where each π_j is a pair $\langle i_j, \sigma_j \rangle$ where $i_j \in [0..n]$ and σ_j is a permutation of $[m]$. We refer to each π_j as an instruction of P. The width of P is m and the length of P is ℓ. For any $\alpha \in \{0,1\}^n$ we define $P(\alpha)$ as follows: fix $\sigma = \mathrm{id}$, and for every $j = 1 \dots s$, we set σ to $\sigma\sigma_j$ if $\pi_j = \langle 0, \sigma_j \rangle$ or $\pi_j = \langle i_j, \sigma_j \rangle$ where $\alpha_{i_j} = 1$, and leave σ unchanged otherwise (that is, if $\pi_j = \langle i_j, \sigma_j \rangle$ where $\alpha_{i_j} = 0$). Our output is the final value of σ. We say that P computes f if there exists a permutation $\sigma^* \neq \mathrm{id}$ such that $P(\alpha) = \mathrm{id}$ if $f(\alpha) = 0$ and $P(\alpha) = \sigma^*$ if $f(\alpha) = 1$. We make three observations about our choices in Definition17. First, the restriction that $f(0 \dots 0) = 0$ will be a convenience; we can always compute $\neg f$ instead if this condition does not hold, or change Definition17 such

that $P(\alpha) = \mathrm{id}$ if $f(\alpha) = 1$ and vice versa.[1] Second, in a layer $\langle i, \sigma_j \rangle$ reading variable x_i, we only fix a permutation in the case that x_i is set to1. This is without loss of generality, as adding a layer of the form $\langle 0, \sigma'_j \rangle$ before an instruction can be thought of as choosing a permutation in the case that x_i is set to0 (while the permutation for $x_i = 1$ can be adjusted accordingly). Before going on to our third observation, we state and prove four simple lemmas which will allow us to conveniently restructure our programs P.

Lemma18. Let P be a permutation branching program computing f and let j be such that $i_j = i_{j+1}$. Then the program P' resulting from replacing π_j, π_{j+1} with $\pi'_j = \langle i_j, \sigma_j \sigma_{j+1} \rangle$ is also a valid program for computing f.

Proof. In both P and P', the permutations σ_j and σ_{j+1} are both applied when $i_j = 1$ and neither are applied when $i_j = 0$. □

Lemma19. Let P be a permutation branching program computing f and let j be such that $\sigma_j \sigma_{j+1} = \sigma_{j+1} \sigma_j$. Then the program P' resulting from switching the order of π_j and π_{j+1} is also a valid program for computing f.

Proof. Consider any assignment α to $\mathbf{x}$. In the case that either α_{i_j} or $\alpha_{i_{j+1}}$ is set to0, these programs compute identical permutations as either σ_j or σ_{j+1} will not be applied. If both are set to1, then

$$P'(\alpha) = \Sigma_1 \sigma_{j+1} \sigma_j \Sigma_2 = \Sigma_1 \sigma_j \sigma_{j+1} \Sigma_2 = P(\alpha)$$

where Σ_1, Σ_2 are the permutations corresponding to the rest of the instructions on input α. □

Lemma20. Let $P = \pi_1 \dots \pi_s$ be a permutation branching program computing f, let $\pi_j = \langle i_j, \sigma_j \rangle$ for all j, and let $j^* \in [\ell]$ be such that $i_{j^*} = 0$. Then there exists a permutation branching program $P' = \pi'_1 \dots \pi'_{j^*-1} \pi'_{j^*+1} \dots \pi'_\ell \pi_{j^*}$ computing f, where $\pi'_j = \langle i_j, \sigma'_j \rangle$ for some permutation σ'_j.

Proof. For $j < j^*$ define $\sigma'_j = \sigma_j$, and for $j > j^*$ define $\sigma'_j = \sigma_{j^*} \sigma_j \sigma_{j^*}^{-1}$. Clearly because $\sigma_{j^*}^{-1}$ and σ_{j^*} cancel out for every adjacent pair of permutations σ'_j, $P'(\alpha)$ contains exactly the same permutations as $P(\alpha)$ in the same order regardless of the assignment α.[2] □

Our next observation is that the layers of the form $\langle 0, \sigma_j \rangle$ are only a convenience and are not necessary to our definition. Let P be our program for f and let $\sigma_1 \dots \sigma_k$ be the permutations corresponding to instructions π_j where $i_j = 0$. By our restriction that $f(0 \dots 0) = 0$ we get $\sigma_1 \dots \sigma_k = P(0 \dots 0) = \mathrm{id}$, and by Lemma20 we can move the instructions π_j with $i_j = 0$ to the end of the program, in order, at which point we can simply remove them all using

[1] We also note that if P computes $\neg f$, we can compute f by appending the instruction $\langle 0, (\sigma^*)^{-1} \rangle$ to P. We avoid taking this route because a later observation will allow us to remove these fixed layers, but only when $f(\alpha) = 0$, which would cause our logic to become circular.

[2] This argument actually allows us to move π_{j^*} to any spot in the program we want, but we are content with just moving them to the end, for reasons which will become immediately clear.

Lemma18 as they compose to the identity for any input α. We can also generalize Lemma20 for restrictions of the function f, meaning when we fix the values of some variables and consider the function on the remaining variables. This is simply the observation that fixing variable x_i turns all instructions of the form $\langle i, \sigma_j \rangle$ into fixed layers $\langle 0, \sigma_j \rangle$.

Corollary21. Let $\rho \in \{0, 1, *\}^n$ and let f_ρ be the function f with x_i fixed to $\rho(i)$ wherever $\rho(i) \neq *$. Let $P = \pi_1 \dots \pi_\ell$ be a permutation branching program computing f, let $\pi_j = \langle i_j, \sigma_j \rangle$ for all j, and let $j^* \in [s]$ be such that $i_{j^*} = 0$ or $\rho(i_{j^*}) \neq *$. Then there exists a permutation branching program $P' = \pi'_1 \dots \pi'_{j^*-1} \pi'_{j^*+1} \dots \pi'_\ell \pi_{j^*}$ computing f_ρ, where $\pi'_j = \langle i'_j, \sigma'_j \rangle$ for some permutation σ'_j and $i'_j = i_j$ iff $\rho(i_j) = *$ and0 otherwise.

Proof. Let program P'' be the result of replacing i_j with0 in each instruction $\pi_j \in P$ such that $\rho(i_j) \neq *$. Clearly this program computes f_ρ, and so applying Lemma20 to P'' completes the proof. □

Assuming that $f_\rho(0 \dots 0) = 0$, by our previous observation this allows us to remove all layers that read variables fixed by ρ. We also note that the other three lemmas hold for f_ρ with no changes. Finally, when f is the AND function, we can rotate our program by moving the first instruction to the end as many times as we like:

Lemma22. Let $P = \Pi_1, \Pi_2$ be a permutation branching program computing $\text{AND}(x_1, \dots, x_n)$, where Π_1 and Π_2 are sequences of instructions. Then $P' = \Pi_2, \Pi_1$ also computes $\text{AND}(x_1, \dots, x_n)$.

Proof. Let $\sigma^* = P'(1, \dots, 1)$ (that is, P' applied to the all-ones input). Referring to Definition17, to prove that P' computes AND, we must show(a) that $\sigma^* \neq \text{id}$,(b) that $P'(\alpha) = \text{id}$ whenever $\text{AND}(\alpha) = 0$, and(c) that $P'(\alpha) = \sigma^*$ whenever $\text{AND}(\alpha) = 1$. (c) follows from the fact that the only string α satisfying $\text{AND}(\alpha) = 1$ is the all-ones string. Now, let $\alpha \in \{0, 1\}^n$. Note that $P(\alpha) = \Pi_1(\alpha)\Pi_2(\alpha)$. If $\text{AND}(\alpha) = 0$, then $P(\alpha) = \text{id}$, so $\Pi_1(\alpha) = (\Pi_2(\alpha))^{-1}$, and so $P'(\alpha) = \text{id}$ as well: this proves(b). Finally, note that $P(1, \dots, 1) \neq \text{id}$, so $\Pi_1(1, \dots, 1) \neq (\Pi_2(1, \dots, 1))^{-1}$, and so $\sigma^* = P'(1, \dots, 1) \neq \text{id}$, proving(a). □

Theorem3 (original comparison statement). For every function f, there is a read-4 permutation branching program of width $2^{n+2^{n-1}}$ and length $4n - 4$ computing f.

Theorem4. Any permutation branching program computing $\text{AND}(x_1, \dots, x_n)$ which reads some variable at most twice must have length at least $4n - 4$.

4.3 Lower bounds

In this section we show that if one tries to get a program of length less than $3n$, one cannot beat Theorem3.

Proof of Theorem4. Let $P = \pi_1\pi_2 \dots \pi_s$ be any program computing $\text{AND}(x_1, \dots, x_n)$. We will write σ_i^j to refer to the permutation in the jth instruction in P that reads variable x_i; in other words, the instructions corresponding to x_i will be $\langle i, \sigma_i^1\rangle \dots \langle i, \sigma_i^k\rangle$ for some k.

Claim27. Any program P computing AND of more than one variable must read every variable at least twice.

Proof. Assume that some variable x_i is read only once in P. Then setting $x_{i'} = 0$ for all $i' \neq i$, we get $\sigma_i^1 = P(0, \dots, 0, 1, 0, \dots, 0) = \text{id}$. Therefore P acts identically whether x_i is0 or1, which is a contradiction because AND depends on x_1. □

Now consider when some variable x_i is read exactly twice. Then P has the form:

$$\Sigma_1, \langle i, \sigma_i^1\rangle, \Pi_1, \langle i, \sigma_i^2\rangle, \Sigma_2.$$

Rotate this program to produce

$$P' = \langle i, \sigma_i^1\rangle, \Pi_1, \langle i, \sigma_i^2\rangle, \Pi_2$$

where $\Pi_2 = \Sigma_2, \Sigma_1$. By Lemma22, P' also computes AND. The following is our main claim.

Claim28. Every variable besides x_i is read at least once in Π_1, and there is at most one such variable $x_{i'}$ which is not read at least twice in Π_1. The same holds for Π_2.

Proof. First, assume for contradiction that there exists $i' \neq i$ such that $x_{i'}$ does not appear in Π_1. Then if we fix $x_{i''} = 1$ for all $i'' \neq i, i'$, we can apply Corollary21 to move all instructions querying variables other than x_i and $x_{i'}$ to the end of the program, to get an equivalent program of the following form which computes $\text{AND}(x_i, x_{i'})$:

$$\langle i, \sigma_i'^1\rangle, \langle i, \sigma_i'^2\rangle, \Sigma$$

where Σ is a sequence of instructions which do not query x_i. Applying Lemma18, we can replace $\langle i, \sigma_i'^1\rangle, \langle i, \sigma_i'^2\rangle$ with a single instruction $\langle i, \sigma''\rangle$ to we get an equivalent program

$$\langle i, \sigma_i''\rangle, \Sigma$$

which also computes $\text{AND}(x_i, x_{i'})$, which contradicts Claim27 as i is only read once. Next, assume for contradiction that there exist $i' \neq i'' \neq i$ such that i' and i'' appear only once each in Π_1. If we fix $x_{i'''} = 1$ for all $i''' \neq i, i', i''$, applying Lemmas21 and18, without loss of generality the following program computes $\text{AND}(x_i, x_{i'})$:

$$\langle i, \sigma_i'^1\rangle, \langle i', \sigma_{i'}\rangle, \langle i'', \sigma_{i''}\rangle, \langle i, \sigma_i'^2\rangle, \Sigma$$

where $\sigma_{i'}$ and $\sigma_{i''}$ are some permutations and Σ is a set of instructions reading only the variables $x_{i'}$ and $x_{i''}$. Define $\Sigma_{i'}$ to be the result of fixing $x_{i''} = 0$ in Σ, and define $\Sigma_{i''}$ to be the result of fixing $x_{i'} = 0$ in Σ. Note that if only one remaining variable is set to1 then the program must output0, so $\sigma_i'^2 = (\sigma_i'^1)^{-1}$,

$\Sigma_{i'} = (\sigma_{i'})^{-1}$, and $\Sigma_{i''} = (\sigma_{i''})^{-1}$. Thus if we set $x_{i''} = 0$ our resulting program is

$$\langle i, \sigma_i'^1 \rangle, \langle i', \sigma_{i'} \rangle, \langle i, (\sigma_i'^1)^{-1} \rangle, \langle i', (\sigma_{i'}^1)^{-1} \rangle$$

and so setting $x_i = x_{i'} = 1$ we get that $\sigma_i'^1 \sigma_{i'} (\sigma_i'^1)^{-1} (\sigma_{i'})^{-1} = \mathrm{id}$. Therefore by Lemma19 we can swap the order of these two instructions and get an equivalent program for $\mathrm{AND}(x_i, x_{i'}, x_{i''})$ of the form

$$\langle i', \sigma_{i'} \rangle, \langle i, \sigma_i'^1 \rangle, \langle i'', \sigma_{i''} \rangle, \langle i, (\sigma_i'^1)^{-1} \rangle, \Sigma$$

and similarly by fixing $x_{i'} = 0$ we have $\sigma_i'^1 \sigma_{i''} (\sigma_i'^1)^{-1} (\sigma_{i''})^{-1} = \mathrm{id}$, which by Lemma19 leaves us with the program

$$\langle i', \sigma_{i'} \rangle, \langle i'', \sigma_{i''} \rangle, \langle i, \sigma_i'^1 \rangle, \langle i, (\sigma_i'^1)^{-1} \rangle, \Sigma$$

and applying Lemma18 on our two layers reading i gives us a program which never reads x_i, which is a contradiction. Finally, to prove the claim for Π_2, observe that by Lemma22,

$$P'' = \langle i, \sigma_i^2 \rangle, \Pi_2, \langle i, \sigma_i^1 \rangle, \Pi_1$$

also computes AND, and apply the above argument to P'', with Π_2 playing the role of Π_1. □

By a simple analysis of Claim28, one of two cases must occur for the variables besides x_i: either1) one variable $x_{i'}$ is read at least twice and all other variables are read at least four times or2) two variables $x_{i'}, x_{i''}$ are read at least three times and all other variables are read at least four times. This is because a read-2 variable can only be read at most once in each of Π_1 and Π_2, while a read-3 variable will be read at most once in either Π_1 or Π_2. In either of these cases, our branching program must have length at least $4(n-2) + 2 \cdot 2 = 4(n-3) + 2 \cdot 1 + 3 \cdot 2 = 4n - 4$. □

References

[1] Eric Allender, Anna Gál, and Ian Mertz. Dual VP classes. Comput. Complex., 26(3):583–625, 2017.

[2] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in nc1. J. Comput. Syst. Sci., 38(1):150–164, 1989.

[3] Michael Ben-Or and Richard Cleve. Computing algebraic formulas using a constant number of registers. SIAM J. Comput., 21(1):54–58, 1992.

[4] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: catalytic space. In Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014, pages 857–866. ACM, 2014.

[5] James Cook and Ian Mertz. Catalytic approaches to the tree evaluation problem. In Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020, pages 752–760. ACM, 2020.

[6] James Cook and Ian Mertz. Encodings and the tree evaluation problem. Electron. Colloquium Comput. Complex., page 54, 2021. URL: https://eccc.weizmann.ac.il/report/2021/054.

[7] Stephen A. Cook, Pierre McKenzie, Dustin Wehr, Mark Braverman, and Rahul Santhanam. Pebbles and branching programs for tree evaluation. ACM Trans. Comput. Theory, 3(2):4:1– 4:43, 2012.

[8] Vincent Girard, Michal Koucky, and Pierre McKenzie. Nonuniform catalytic space and the direct sum for space. Electronic Colloquium on Computational Complexity (ECCC), 138, 2015.

[9] Frank Gray. Pulse code communication. https://patents.google.com/patent/US2632058A/ en, 1953. US Patent 2632058A.

[10] E.I. Nečiporuk. A boolean function. Dokl. Akad. Nauk SSSR, 169(4), 1966.

[11] Aaron Potechin. A note on amortized branching program complexity, 2017.

[12] Robert Robere and Jeroen Zuiddam. Amortized circuit complexity, formal complexity measures, and catalytic algorithms. In FOCS, pages 759–769. IEEE, 2021.